System Analysis & Design

1. Problem Statement & Objectives

Description

Student academic performance is influenced by a combination of academic, behavioral, and socio-economic factors. These include study habits (e.g., hours studied), attendance levels, motivation, parental involvement, and family income.

However, despite the availability of such data, there is often no structured or systematic approach to analyze these variables together and determine their relative impact on student outcomes. Most evaluations rely on isolated observations rather than comprehensive data-driven analysis.

Problem Statement

There is a lack of a structured SQL-based analytical system that can efficiently process student performance data, analyze multiple influencing factors simultaneously, and generate clear, data-driven insights about what drives academic success.

Objectives

- To load and manage student performance data using **SQL Server**
- To analyze key variables such as:
    - Hours_Studied
    - Attendance
    - Parental_Involvement
    - Motivation_Level
    - Family_Income
    - Previous_Scores
- To perform **multi-factor analysis** combining multiple variables instead of analyzing them independently

- To segment students into:
    - High Performers
    - Medium Performers
    - Low Performers

- To identify patterns such as:
  - “High attendance leads to better performance”
  - “Motivation enhances the effect of study hours”
- To implement **stored procedures** for reusable and repeatable analysis
- To generate clear insights supported by SQL queries and Excel visualizations

2. Use Case Diagram & Description

Description

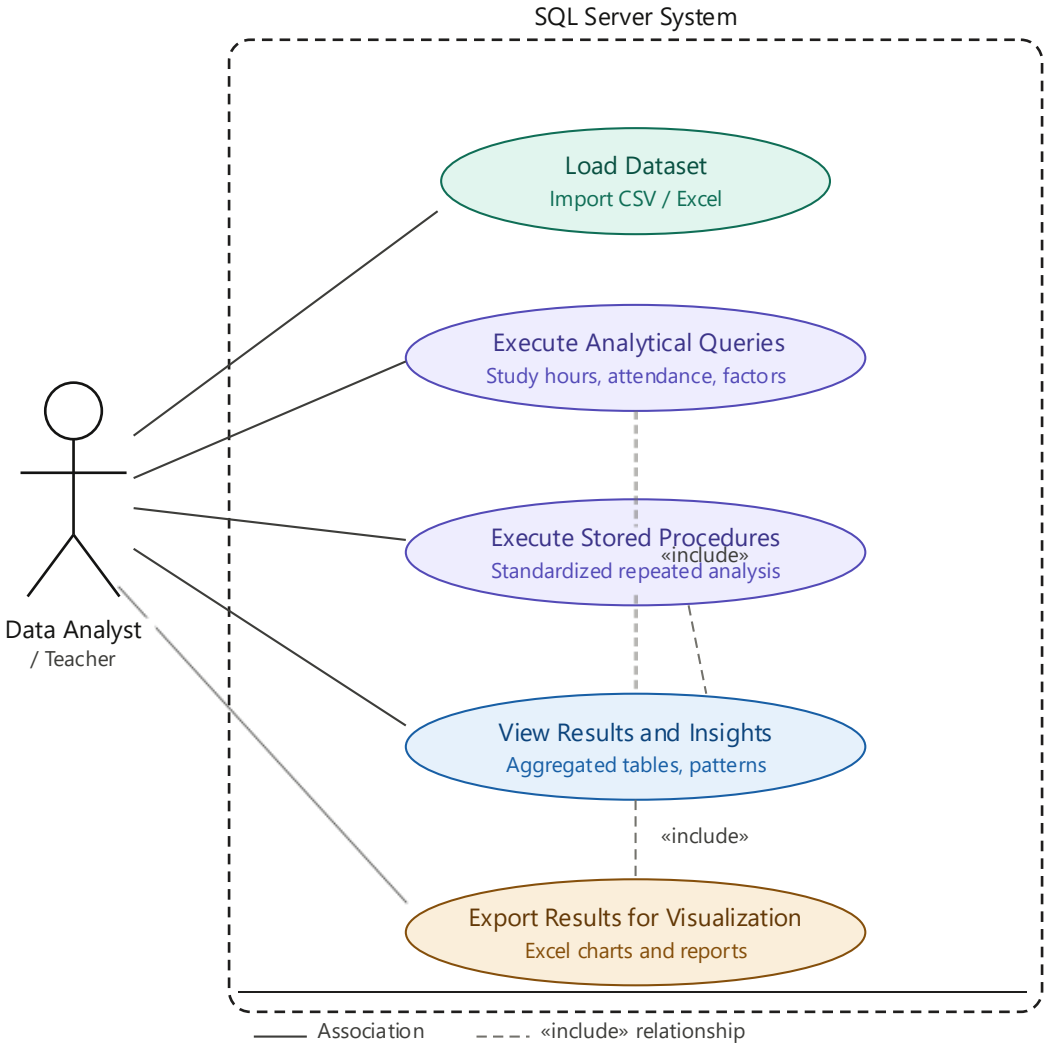
The system is designed with minimal user interaction, as it primarily focuses on SQL-based data analysis rather than a full interactive software application. The user interacts directly with the database by executing queries and retrieving analytical results.

Primary Actor

- Data Analyst / Teacher

Role:

The primary user who interacts with the system to analyze student performance data and extract insights.



Main Use Cases:

1. Load Dataset

Description:
The user imports the student performance dataset into Microsoft SQL Server.

Process:

- Upload dataset (CSV/Excel)
- Store data in a structured table
- Ensure data is properly formatted

2. Execute Analytical Queries

Description:
The user runs SQL queries to analyze relationships between different variables.

Examples:

- Study hours vs exam score
- Attendance vs performance
- Multi-factor analysis (e.g., motivation + attendance)

3. Execute Stored Procedures

Description:
The user runs predefined stored procedures to perform repeated or complex analysis efficiently.

Purpose:

- Save time
- Standardize analysis
- Reduce manual query writing

4. View Results and Insights

Description:
The system returns structured results based on executed queries.

Output:

- Tables with aggregated results
- Identified patterns and relationships
- Data ready for visualization in Excel

5. Export Results for Visualization

Description:
The user exports query results to Excel for chart creation and reporting.

3. Functional & Non-Functional Requirements

Functional Requirements

Description

These define the core capabilities of the SQL-based analysis system.

The system should be able to:

Data Management

- Load the dataset into SQL Server
- Store student data in a structured relational table
- Handle missing or inconsistent values during preprocessing

Data Analysis

- Execute analytical SQL queries on the dataset
- Perform aggregation operations:
    - AVG (average scores)
    - COUNT (student distribution)
    - GROUP BY (categorical analysis)

Relationship Analysis

- Analyze relationships between variables such as:
    - Hours_Studied vs Exam_Score
    - Attendance vs Performance
    - Motivation_Level combined with study habits

Multi-Factor Analysis

- Combine multiple variables in a single query to identify deeper patterns
- Detect interactions between factors (e.g., motivation + attendance)

Student Segmentation

- Classify students into:
  - High Performers
  - Medium Performers
  - Low Performers
- Compare characteristics across these groups

Automation

- Implement stored procedures for:
  - Repeated analysis
  - Generating summarized reports
- Enable easy execution without rewriting queries

Output Generation

- Produce structured query results (tables)
- Export results to Excel for visualization
- Support creation of charts (e.g., trends and comparisons)

Non-Functional Requirements

Description

These define system quality attributes and constraints to ensure reliability and performance.

Performance

- Queries should execute efficiently on datasets ranging from 1k to 10k records
- Avoid unnecessary complexity or redundant operations

Accuracy

- All calculations (averages, comparisons, segmentation) must be correct
- Results should be validated through multiple query approaches

Scalability

- The system should handle increased data size with minimal modifications
- SQL structure should support extension to larger datasets

Reliability

- Queries and stored procedures must execute consistently without failure
- Results should be reproducible across multiple runs

Maintainability

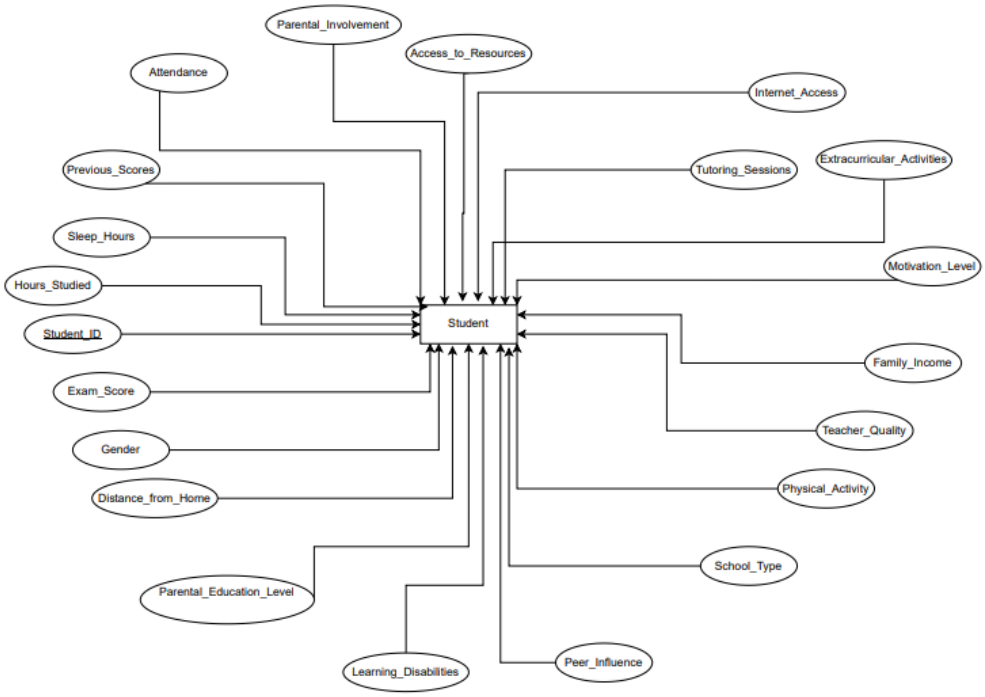
- SQL scripts should be clean, readable, and well-structured
- Stored procedures should be easy to update or extend

Usability

- Output results should be clear and easy to interpret
- Insights should be understandable for non-technical stakeholders (e.g., teachers)

4.Database Design & Data Modeling

- **ER Diagram**



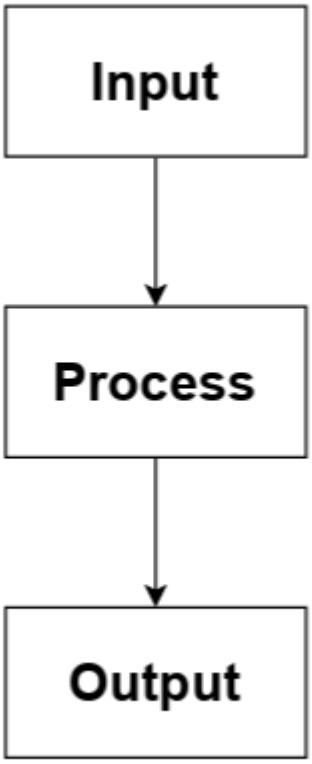
- **Logical Schema**

Student
Student_ID (PK) ---------- Hours_Studied Attendance Parental_Involvement Access_to_Resources Extracurricular_Activities Sleep_Hours Previous_Scores Motivation_Level Internet_Access Tutoring_Sessions Family_Income Teacher_Quality School_Type Peer_Influence Physical_Activity Learning_Disabilities Parental_Education_Level Distance_from_Home Gender Exam_Score

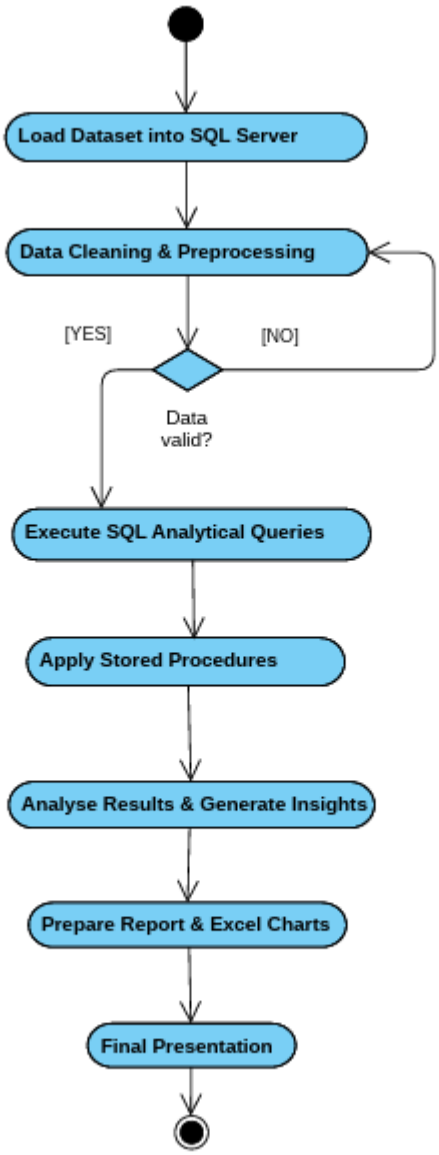
- **Physical Schema**

```
CREATE TABLE Student (
  Student_ID INT PRIMARY KEY,
  Hours_Studied FLOAT,
  Attendance FLOAT,
  Parental_Involvement VARCHAR(20),
  Access_to_Resources VARCHAR(20),
  Extracurricular_Activities VARCHAR(10),
  Sleep_Hours INT,
  Previous_Scores INT,
  Motivation_Level VARCHAR(20),
  Internet_Access VARCHAR(10),
  Tutoring_Sessions INT,
  Family_Income VARCHAR(20),
  Teacher_Quality VARCHAR(20),
  School_Type VARCHAR(20),
  Peer_Influence VARCHAR(20),
  Physical_Activity INT,
  Learning_Disabilities VARCHAR(10),
  Parental_Education_Level VARCHAR(30),
  Distance_from_Home VARCHAR(20),
  Gender VARCHAR(10),
  Exam_Score FLOAT
);
```

5. DFD (Data Flow Diagram)



6. Activity Diagram



7. Technology Stack

Description

This section defines the technologies and tools used to implement the student performance analysis system. The project is primarily built on SQL for data storage, processing, and analysis, with Excel used for visualization and reporting.

Components

Database: Microsoft SQL Server

Role:

The core component of the system used for data storage, management, and analysis.

Usage in Project:

- Storing the student performance dataset in a structured relational table
- Executing analytical SQL queries (e.g., aggregation, grouping, filtering)
- Performing multi-factor analysis across variables
- Implementing **stored procedures** for reusable and automated analysis
- Ensuring data integrity and efficient query execution

Query Language: SQL (Structured Query Language)

Role:

The primary language used for interacting with the database.

Usage in Project:

- Writing queries for:
    - Data exploration
    - Aggregation (AVG, COUNT, etc.)
    - Relationship analysis (e.g., Hours_Studied vs Exam_Score)
- Performing segmentation of students (High / Medium / Low performers)
- Developing stored procedures for repeated analysis

Visualization & Reporting: Microsoft Excel

Role:

Used to present analysis results in a visual and interpretable format.

Usage in Project:

- Exporting SQL query results into Excel
- Creating charts such as:
  - Study hours vs average score
  - Attendance vs performance groups
- Supporting clear communication of insights

Explanation

The project is centered around **SQL-based data analysis**, where Microsoft SQL Server handles all data-related operations, including storage, querying, and analysis.

SQL is used to extract meaningful insights through aggregation, multi-variable analysis, and segmentation techniques.

Microsoft Excel complements the system by providing simple but effective visualizations, enabling stakeholders to better understand patterns and trends identified through SQL queries.

8. Testing & Validation

Description

This phase ensures the correctness, reliability, and consistency of the SQL-based analysis system. It focuses on validating that all queries, calculations, and insights accurately reflect the dataset and produce logically sound results.

Testing & Validation Approach

1. Logical Validation of Query Results

Description:

Each query result is evaluated against expected logical behavior based on the dataset.

Implementation:

- Verify that:
  - Higher Hours_Studied generally corresponds to higher Exam_Score
  - Higher Attendance leads to improved performance
- Ensure that results align with realistic academic expectations

2. Cross-Verification Using Multiple Query Methods

Description:

Critical results are validated using different SQL approaches to ensure consistency.

Implementation:

- Compare results using:
  - GROUP BY aggregation vs subqueries
  - Direct queries vs stored procedures
- Ensure outputs remain consistent across methods

3. Data Consistency & Anomaly Detection

Description:

Identify and handle irregular or inconsistent data that may affect analysis.

Implementation:

- Detect anomalies such as:
  - Extremely high/low values (outliers)
  - Missing or NULL values
- Apply:
  - Data cleaning techniques
  - Standardization where necessary

4. Segmentation Validation

Description:

Ensure correctness of student classification into performance groups.

Implementation:

- Verify segmentation logic:
  - High Performers
  - Medium Performers
  - Low Performers
- Check that:
  - Each student falls into exactly one category
  - No overlap or misclassification occurs

5. Stored Procedure Testing

Description:

Ensure that stored procedures execute correctly and return expected results.

Implementation:

- Execute each stored procedure multiple times

- Validate output consistency
- Test with different conditions if applicable

6. Query Execution Stability

Description:

Ensure that all SQL queries run without errors under normal conditions.

Implementation:

- Test queries on full dataset
- Confirm:
  - No runtime errors
  - No syntax issues
  - Stable execution across runs
  -

7. Result Accuracy Verification

Description:

Ensure numerical outputs are correct and precise.

Implementation:

- Manually verify sample calculations
- Check:
  - Averages (AVG)
  - Counts (COUNT)
  - Grouped results